

Question 10

In the developer productivity tooling, the agent is asked to handle two incoming requests: a clear off-by-one defect in a single function with an obvious stack trace, and a migration from one HTTP client library to another across more than 45 files with several possible implementation strategies. How should the team assign plan mode and direct execution?

This question deals with the [Developer Productivity with Claude Exam Scenario](#).

(Choose 1)

Use plan mode for both requests because additional planning is always beneficial

Use direct execution for both requests because plan mode is only useful when starting a new project

Use direct execution for the single-function bug fix, and use plan mode for the migration because its scale and competing approaches need exploration before implementation ✓

Use plan mode for the bug fix, and use direct execution for the migration

ANSWER

The correct option is **Use direct execution for the single-function bug fix, and use plan mode for the migration because its scale and competing approaches need exploration before implementation.**

This option is correct because a small, well scoped defect with a clear stack trace and a single function change is a classic case for direct execution. The task is bounded, the failure mode is known, and the feedback loop is fast, so immediate action reduces overhead and delivers value quickly. In contrast, a library migration that touches more than 45 files introduces architectural choices, dependency nuances, and sequencing concerns. Plan mode lets the agent surface strategies, compare tradeoffs, outline code changes, and de-risk the rollout before any bulk edits begin.

Plan mode also helps choose the right migration path, such as adapter patterns, automated refactors, or phased rollouts with guardrails and tests. It clarifies ordering, identifies shared abstractions, and calls out cross-cutting impacts like retries, timeouts, and error handling differences between clients. That upfront structure accelerates safe execution and reduces rework.

Use plan mode for both requests because additional planning is always beneficial is incorrect because planning adds cost without benefit when the task is trivial and already well understood. For a single clear bug, added ceremony delays the fix and does not improve quality.

Use direct execution for both requests because plan mode is only useful when starting a new project is incorrect because plan mode is valuable any time scope is large or ambiguous, not only for greenfield work. A multi file migration benefits from explicit strategy and risk management before code changes begin.

Use plan mode for the bug fix, and use direct execution for the migration is incorrect because it inverts the appropriate levels of structure. The simple bug needs a quick change, while the broad migration needs deliberate planning.

EXAM TIP

Map the mode to the work. Choose *direct execution* when the task is small, localized, and the path is clear. Choose *plan mode* when scope is wide, dependencies vary, or multiple approaches must be compared before changing code.

Question 9

In the customer support resolution agent, a billing dispute has taken more than 25 turns to investigate, and the agent has determined that a payment gateway timeout caused retry logic to create duplicate charges. The needed refund is \$847, which is above the agent's \$500 process_refund authorization limit, so it must use escalate_to_human, and the human reviewer will not receive the full transcript. What context should the agent include for effective handoff?

This question deals with the [Customer Support Resolution Agent Exam Scenario](#).

(Choose 1)

The customer's initial complaint copied verbatim along with selected tool output snippets that show the duplicate charges

A structured handoff containing the customer ID, cause of the issue, refund amount, and proposed next action ✓

The entire conversation history including every MCP tool request and response from the investigation

The agent's final diagnosis and the refund amount

ANSWER

The correct option is **A structured handoff containing the customer ID, cause of the issue, refund amount, and proposed next action**. This provides the reviewer with the essential who, what happened, how much, and what to do next even though they will not see the full transcript.

A structured handoff is best because the reviewer needs a concise and complete summary when the conversation has run long and the full history is unavailable. The agent should include the customer ID so the reviewer can immediately find the right account. It should capture the root cause that a payment gateway timeout triggered retry logic and created duplicate charges so the reviewer understands why the refund is justified. It should include the exact refund amount of 847 to enable a clear decision. It should also propose the next action such as approve the refund and record the resolution since the agent lacks authorization above 500. This format reduces back and forth, speeds approval, and supports auditability.

The customer's initial complaint copied verbatim along with selected tool output snippets that show the duplicate charges is not sufficient because it is unstructured and noisy. It may omit key identifiers, the exact amount to refund, and the recommended next step, so the reviewer would still need clarification.

The entire conversation history including every MCP tool request and response from the investigation is inappropriate because the reviewer will not receive the full transcript and it is also inefficient. Long transcripts and raw tool logs slow decision making and often contain details that are not needed for approval.

The agent's final diagnosis and the refund amount is incomplete because it omits the customer ID and the proposed next action. It also fails to make the handoff actionable for a reviewer who must know exactly what to approve and how to proceed.

EXAM TIP

When a reviewer will not see the full conversation, favor a *structured handoff* that captures *who*, *what happened*, *how much*, and the *next action*. Keep it concise and include only the fields needed to make a decision.

REFERENCES

Check out these links for more details:

- [Agents for Amazon Bedrock](#)
- [What is Amazon Connect](#)

Question 8

The customer support resolution agent uses a fixed returns flow: first determine return eligibility from customer and order data, then issue return approval, then create a shipping label, and finally update inventory. Each stage depends on the prior result, and if the return is ineligible, all later stages must be bypassed. Tests show the agent is still creating labels for ineligible returns. Which workflow pattern best fits this situation?

This question deals with the [Customer Support Resolution Agent Exam Scenario](#).

(Choose 1)

Orchestrator-Worker — Have a central LLM assess the return case and dynamically assign subtasks to worker agents

Prompt Chaining — Split the flow into ordered dependent stages with intermediate gates before later stages run ✓

Routing — Send the case to either an approval workflow or a rejection workflow after eligibility is determined

Parallelization — Run eligibility, approval, label creation, and inventory update at the same time, then ignore unused outputs

ANSWER

The correct option is **Prompt Chaining — Split the flow into ordered dependent stages with intermediate gates before later stages run** because the scenario requires a fixed sequence where each step depends on the previous result and downstream steps must be blocked when eligibility fails.

This pattern splits the returns flow into discrete prompts that execute in a fixed order and it inserts an explicit decision point before each subsequent step runs. Each stage consumes the structured output of the prior stage and only proceeds when the gate indicates success, which prevents label creation and inventory updates when a return is ineligible.

By chaining the stages and carrying forward a clear eligibility and approval status, the workflow guarantees that side effect steps like label generation and inventory updates only occur after all prerequisites have passed. This aligns with the requirement to bypass all later stages when an earlier condition fails and it directly resolves the test failure in the scenario.

Orchestrator-Worker — Have a central LLM assess the return case and dynamically assign subtasks to worker agents is not the best fit because the flow is not dynamic and the steps are predetermined, so there is no need for task decomposition or flexible ordering. This approach does not inherently enforce hard gates that prevent later actions when eligibility is false, so it would not directly address labels being created for ineligible returns.

Routing — Send the case to either an approval workflow or a rejection workflow after eligibility is determined is insufficient because the process needs multiple dependent steps after approval and requires checks between each stage. A single route into either approval or rejection does not provide the step by step gating that would stop label creation and inventory updates when any prerequisite fails.

Parallelization — Run eligibility, approval, label creation, and inventory update at the same time, then ignore unused outputs is incorrect because the stages are dependent and must not run concurrently. Running them together would create exactly the observed bug where labels are generated even when returns are ineligible.

EXAM TIP

When a scenario highlights *ordered* dependent steps and strict *gates* that must block later actions on failure, prefer prompt chaining. Reserve orchestrator worker for *dynamic* task assignment and use parallelization only when tasks are truly *independent*.

Question 7

In the multi-agent research system, the coordinator asks several research subagents to independently investigate the same topic so the final report is more reliable. Production review shows that some weakly supported claims raised by only one subagent are still being carried into reports unchanged. The team wants a threshold rule that marks a finding for review only when more than one subagent raises the same concern. Which Parallelization pattern best fits this design?

This question deals with the [Multi-Agent Research Exam Scenario](#).

(Choose 1)

Prompt Chaining — Combine each subagent's output in sequence, giving the next subagent the prior output as context

Voting — Run multiple subagents on the same task independently and use an agreement threshold to determine the accepted result ✓

Orchestrator-Workers — Have the coordinator evaluate each subagent's output during execution and dynamically decide whether to accept it

Sectioning — Split the research topic into separate subtopics and assign each subagent a different section to investigate in parallel

ANSWER

The correct option is **Voting — Run multiple subagents on the same task independently and use an agreement threshold to determine the accepted result.**

This pattern matches the requirement to only mark a finding when more than one subagent raises the same concern. It runs several agents independently on the same prompt and then applies a consensus rule such as a k of n threshold. That way a single weakly supported claim fails to meet the threshold and is flagged for review or filtered out, while repeated signals from multiple agents are accepted. Independence reduces cross-contamination and makes the threshold meaningful, which directly addresses the team's need to prevent solo claims from passing through unchanged.

Prompt Chaining — Combine each subagent's output in sequence, giving the next subagent the prior output as context is not correct because it is a sequential dependency pattern rather than a parallel agreement mechanism. It tends to propagate earlier statements into later steps and can amplify a single weak claim instead of requiring independent corroboration.

Orchestrator-Workers — Have the coordinator evaluate each subagent's output during execution and dynamically decide whether to accept it is not correct because it relies on a central controller's judgment rather than peer agreement among independent agents. It does not implement an explicit agreement threshold across agents, so a single subagent's claim might still be accepted by the orchestrator.

Sectioning — *Split the research topic into separate subtopics and assign each subagent a different section to investigate in parallel* is not correct because it divides the scope instead of having multiple agents investigate the same question. Since each agent works on a different slice, there is no basis for cross-agent agreement on the same claim, so a threshold rule cannot be applied to corroborate findings.

EXAM TIP

When a question mentions independent agents working on the same task and an *agreement threshold* to accept results, map it to a consensus pattern like *voting* rather than sequential refinement or topic splitting.

REFERENCES

Check out these links for more details:

- [Anthropic docs: Prompt engineering](#)
- [Anthropic Cookbook: Examples and patterns for building with Claude](#)

Question 6

While integrating Claude Code into the team's development workflow, you notice generated database migrations use mixed names such as 0007_add_index.sql and add-index-2024-04.sql, which makes ordering fragile. The team wants every generated migration file to use a timestamp format like 20240418_091530_add_index.sql. What is the most maintainable way to enforce this requirement?

This question deals with the [Code Generation with Claude Code Exam Scenario](#).

(Choose 1)

Add guidance to CLAUDE.md that migration files must use the YYYYMMDD_HHMMSS_name.sql naming pattern

Define a dedicated migration-generation skill and ask developers to invoke it through a /migrate command

Configure a PreToolUse hook that checks file creation requests and blocks migration filenames that do not match the required pattern ✓

Create a .claude/rules/ file with frontmatter paths: ["db/migrations/*.sql"] that describes the filename convention, template, and validation steps

ANSWER

The correct answer is **Configure a PreToolUse hook that checks file creation requests and blocks migration filenames that do not match the required pattern**. This approach enforces the timestamped naming convention at the moment a migration file is created and ensures consistent ordering across the repository.

This hook intercepts proposed file writes before they land in the codebase. You can validate filenames against a strict pattern such as YYYYMMDD_HHMMSS_name.sql and reject anything that does not match while returning a clear correction message. Because the validation runs automatically and centrally, it removes reliance on human memory, prevents drift across contributors and tools, and provides a single maintainable control point as the policy evolves.

Add guidance to CLAUDE.md that migration files must use the YYYYMMDD_HHMMSS_name.sql naming pattern is incorrect because documentation alone does not enforce behavior. Developers and the model can still generate files with inconsistent names, and reviews may miss violations, which makes this unreliable and harder to maintain.

Define a dedicated migration-generation skill and ask developers to invoke it through a /migrate command is incorrect because it depends on people remembering to use the command and it can be bypassed by direct edits or other generators. This also fragments the workflow and adds ongoing maintenance without guaranteeing consistent outcomes.

Create a `.claude/rules/` file with frontmatter paths: `["db/migrations/*.sql"]` that describes the filename convention, template, and validation steps is incorrect because a descriptive rules file does not provide an enforcement point. Without a blocking control tied to file creation, noncompliant filenames would still slip through.

EXAM TIP

When you compare options, favor controls that provide *preventive* and *automated* enforcement at the point of change over guidance that relies on memory or manual reviews.

Question 5

In the structured data extraction system, the team is designing a JSON schema field for payment terms. A sample of 50 documents shows that 70% contain standard terms such as "Net 30" or "Net 60," 20% contain arrangements such as "three installments" or "50% upfront and 50% on delivery," and 10% contain unresolved wording such as "to be discussed separately" or "to be determined after contract execution." An initial enum containing only "net_30," "net_60," and "net_90" caused 30% of the extracted values to be collapsed into "unknown." How should the field be modeled?

This question deals with the [Structured Data Extraction Exam Scenario](#).

(Choose 1)

Use an enum with only "net_30," "net_60," and "net_90," and map every other payment term to "unknown" to keep the field tightly structured

Use only a numeric field for days until payment, and skip documents that describe installment plans or unresolved payment conditions

Use a free-text field that stores the extracted payment term exactly as it appears in the source document

Use an enum for common values such as "net_30," "net_60," and "net_90," add "other," and require a detail field when "other" is selected ✓

ANSWER

The correct option is **Use an enum for common values such as "net_30," "net_60," and "net_90," add "other," and require a detail field when "other" is selected.**

This approach preserves structure where it matters most and still captures the richness of real documents. The sample shows that about seventy percent are standard net terms, so enumerating common values gives clean analytics and validation. Adding an other bucket with a required detail field ensures you do not lose information for installment plans and unresolved language. This reduces the number of collapsed values and improves downstream usability because analysts can still group on the enum and inspect the detail text for the remainder. It also supports future growth since you can promote frequently recurring values from the detail field into the enum later without breaking existing records.

Use an enum with only "net_30," "net_60," and "net_90," and map every other payment term to "unknown" to keep the field tightly structured is incorrect because it collapses roughly thirty percent of cases into a single unknown bucket. That discards important distinctions between installment plans and unresolved terms, reduces traceability, and weakens analytics and quality control.

Use only a numeric field for days until payment, and skip documents that describe installment plans or unresolved payment conditions is incorrect because a single number cannot represent installments

or split payments and skipping records introduces bias and data loss. The model should capture reality rather than discard nonconforming but valid business terms.

Use a free-text field that stores the extracted payment term exactly as it appears in the source document is incorrect because it removes structure that is needed for validation and aggregation. Free text prevents reliable grouping and filtering for the common net terms and it leads to inconsistent phrasing that makes analytics harder.

EXAM TIP

When modeling semi-structured fields, prefer a *hybrid* pattern. Enumerate the common normalized values for easy grouping, add an *other* option to avoid data loss, and require a concise *detail* field so you retain meaning for nonstandard cases.

REFERENCES

Check out these links for more details:

- [JSON Schema validation keywords: enum](#)
- [JSON Schema conditionals with if, then, and else for requiring details](#)
- [JSON Schema basics and design considerations](#)

Question 4

In the multi-agent research system, a third-party academic paper API used by the paper retrieval tool changed its authentication scheme, causing the existing key to fail. The tool now receives HTTP 401 Unauthorized responses, but the agent treats them as missing papers and the report generator produces incomplete cited reports; 76 reports were affected over the last two days. What is the best way to address the underlying reliability problem?

This question deals with the [Multi-Agent Research Exam Scenario](#).

(Choose 1)

Add a system prompt instruction telling the agent that HTTP 401 means an authentication failure rather than a paper-not-found result

Remove the academic paper API integration and have the research pipeline use the web search subagent for paper retrieval instead

Update the tool implementation to classify HTTP status codes and return 401/403 as structured errors with `is_retryable: false` and `error_category: auth`, allowing the coordinator to route based on that category ✓

Add exponential backoff with up to five retries to every academic paper API request so temporary authentication failures can recover automatically

ANSWER

The correct answer is **Update the tool implementation to classify HTTP status codes and return 401/403 as structured errors with `is_retryable: false` and `error_category: auth`, allowing the coordinator to route based on that category**. This choice directly addresses the root cause by making authentication failures explicit, non retryable, and machine readable so the system stops misclassifying them as missing papers and the coordinator can take the right action.

HTTP 401 and 403 indicate authentication or authorization problems rather than transient or not found conditions. By emitting structured errors with `is_retryable: false` and `error_category: auth`, the tool prevents wasteful retries, avoids silent data quality degradation, and enables the coordinator to route to remediation paths. Those paths can include refreshing credentials, invoking a fallback retrieval method that preserves citation integrity, failing fast with a clear error, or alerting an operator. This approach improves reliability and observability, which stops incomplete cited reports from being produced and makes it easier to detect and correct issues when an upstream API changes its authentication scheme.

Add a system prompt instruction telling the agent that HTTP 401 means an authentication failure rather than a paper-not-found result is not sufficient because it relies on prompt interpretation rather than robust tool level error handling. Prompt guidance can still be misapplied under ambiguous contexts, and it does not provide the coordinator with structured signals to route or alert.

It also fails to mark the condition as non retryable and will not create the auditability needed to prevent quiet data loss.

Remove the academic paper API integration and have the research pipeline use the web search subagent for paper retrieval instead sidesteps the root cause and risks lowering precision and recall for papers, especially for sources that require authenticated access. It also increases variability, latency, and potential cost without fixing the underlying misclassification and lack of structured error handling that caused the incomplete reports.

Add exponential backoff with up to five retries to every academic paper API request so temporary authentication failures can recover automatically is inappropriate for 401 and 403 because auth failures are not transient network conditions. Retrying will prolong failures, waste quota, and still produce incomplete outputs. The correct behavior is to treat these as non retryable and surface them to orchestration for clear routing and remediation rather than attempting automated retries.

EXAM TIP

When APIs return HTTP 401 or 403, prefer designs that emit *structured, non-retryable* auth errors so the orchestrator can *route, alert, or fail fast* instead of masking the issue or retrying.

REFERENCES

Check out these links for more details:

- [AWS Builders' Library: Timeouts, retries, and backoff with jitter](#)
- [AWS SDKs: Retry behavior and non-retryable errors](#)
- [AWS Step Functions error handling and routing with Catch and Retry](#)

Question 3

In the customer support resolution agent, `lookup_order` is selected appropriately, but traces show repeated failures because the agent sends phrases such as "the order I placed last Tuesday" where the MCP tool expects an order ID or an ISO-8601 date-time value. Its current description says, "Looks up order information by date or identifier." What tool-definition change would most directly reduce these malformed `lookup_order` calls?

This question deals with the [Customer Support Resolution Agent Exam Scenario](#).

(Choose 1)

Specify accepted input formats, required constraints, and sample `lookup_order` calls in the tool description ✓

Add a system prompt rule telling the agent to use ISO-8601 values whenever it calls `lookup_order`

Add schema validation that rejects nonconforming `lookup_order` inputs and returns an error response

Replace `lookup_order` with `lookup_order_by_id` and `lookup_order_by_date` so each lookup path is separate

ANSWER

The correct choice is **Specify accepted input formats, required constraints, and sample `lookup_order` calls in the tool description**. This directly tells the model exactly what argument shapes are valid and shows concrete examples, which leads to fewer malformed tool invocations.

A precise tool description is the strongest and most local signal the model uses when constructing tool calls. When you clearly state that the date must be an ISO-8601 date-time and that the identifier must be an order ID with an expected pattern, and you include short example calls that demonstrate both paths, the model is far more likely to generate conforming arguments on the first attempt. This reduces retries and prevents phrases like the order I placed last Tuesday from being sent at all, because the model will convert such phrases into the required formats before calling the tool.

Add a system prompt rule telling the agent to use ISO-8601 values whenever it calls `lookup_order` is not the most direct fix because broad system guidance is weaker at call time than specific tool metadata. The failure stems from ambiguity in the tool's own description, so strengthening that description works better than a general reminder that may be overlooked during argument construction.

Add schema validation that rejects nonconforming `lookup_order` inputs and returns an error response guards the tool but it only catches mistakes after they happen. It can improve robustness, yet it does not reduce the initial rate of malformed calls as effectively as making the tool description explicit and example driven.

Replace `lookup_order` with `lookup_order_by_id` and `lookup_order_by_date` so each lookup path is separate adds complexity without addressing the real cause, which is unclear input expectations. Splitting the tool can still result in natural language arguments being passed, and it introduces additional decision points for the model which can increase mis-selection.

EXAM TIP

When a tool is being misused, strengthen the *tool description* first. List exact formats, state constraints, and include short examples so the model can pattern match and produce valid arguments on the first try.

REFERENCES

Check out these links for more details:

- [Anthropic documentation: Tool use with Claude](#)
- [Model Context Protocol: Tools concept](#)

Question 2

The team adds a legacy-document conversion capability to an MCP server used by the developer productivity agent. After several iterations, the server author has a carefully evaluated prompt that produces much better Markdown output than engineers usually get from ad hoc instructions. What is the role of the MCP Prompts primitive in making this optimized workflow available to users?

This question deals with the [Developer Productivity with Claude Exam Scenario](#).

(Choose 1)

It automatically adds the prompt to the system prompt whenever Claude calls any tool from that MCP server

It exposes reusable, evaluated prompt templates for the server's domain so users can run the optimized conversion workflow ✓

It supplies a reusable configuration file for MCP server credentials, environment variables, and connection parameters

It defines the JSON-RPC message structure used for communication between MCP clients and MCP servers

ANSWER

The correct option is **It exposes reusable, evaluated prompt templates for the server's domain so users can run the optimized conversion workflow.**

This is correct because the MCP Prompts primitive lets a server publish named and parameterized prompt templates that clients can discover and run. By exposing the carefully tested conversion prompt through Prompts, users can invoke the conversion workflow consistently without rewriting instructions each time, which leads to higher quality and more predictable Markdown output. Clients such as editors or assistants can list available prompts from a connected server and execute them with inputs, which operationalizes the server author's optimized approach for the whole team.

Prompts are designed for reuse and standardization. They capture proven instruction patterns and make them callable as a first class capability of the server. This makes it easy to maintain a single source of truth for a domain specific workflow, to iterate on it centrally, and to immediately propagate improvements to all users who connect to the server.

It automatically adds the prompt to the system prompt whenever Claude calls any tool from that MCP server is incorrect because Prompts are not auto injected into the model's system prompt and they are not tied to tool invocation. Users or clients explicitly choose when to run a prompt, and tools remain a separate primitive for performing actions.

It supplies a reusable configuration file for MCP server credentials, environment variables, and connection parameters is incorrect because connection and credential configuration belongs to client

or deployment setup, not to the Prompts primitive. Prompts expose reusable instructions, not connection details.

It defines the JSON-RPC message structure used for communication between MCP clients and MCP servers is incorrect because the protocol specification defines message structures and transport. The Prompts primitive operates at the content layer by publishing reusable templates and does not define wire level messaging.

EXAM TIP

When you see language about reusable instruction patterns or evaluated templates, map it to *Prompts*. Reserve *Tools* for actions and side effects, and *Resources* for data exposure. Matching the verb in the scenario to the right primitive quickly leads you to the correct choice.

Question 1

In the structured data extraction pipeline, Claude reliably extracts required fields from documents that present data in tables, but accuracy drops when the same fields appear only in narrative paragraphs. What is the most targeted way to improve extraction from those prose-heavy documents?

This question deals with the [Structured Data Extraction Exam Scenario](#).

(Choose 1)

Strengthen the main prompt with firmer wording about accuracy and more detailed general extraction instructions

Add few-shot examples that demonstrate extracting the required fields from prose passages where values are embedded in sentences ✓

Convert every document into a table-like representation before sending it to Claude for JSON schema validation

Use a separate prose prompt that first finds sentences containing numbers and then extracts fields only from those sentences

ANSWER

The correct option is **Add few-shot examples that demonstrate extracting the required fields from prose passages where values are embedded in sentences**. This choice most directly addresses the model's weakness on narrative text by showing clear precedents that map sentences to the target fields.

Providing targeted few-shot demonstrations teaches the model the precise pattern to follow when values appear inside sentences rather than in cells. Well chosen exemplars anchor the model's attention to the cues that matter in prose such as named entities, temporal phrases, and value qualifiers, and they illustrate exactly how to produce the expected JSON for those cases. This narrows ambiguity, aligns the model with the evaluation criteria, and usually yields the largest precision and recall gains when the failure mode is specific to a format shift from tables to narrative paragraphs.

Strengthen the main prompt with firmer wording about accuracy and more detailed general extraction instructions is not sufficient because stronger language rarely fixes systematic misses caused by different input structure. Generic guidance can help with formatting or completeness, yet it does not provide the concrete pattern learning that the model needs to reliably parse prose-only inputs.

Convert every document into a table-like representation before sending it to Claude for JSON schema validation is heavy handed and risks losing context or introducing preprocessing errors. It adds complexity to the pipeline and does not directly teach the model how to interpret sentences, so it is not the most targeted improvement for this specific accuracy gap.

Use a separate prose prompt that first finds sentences containing numbers and then extracts fields only from those sentences relies on a brittle heuristic. Many required fields may be textual or may not appear in the same sentence as a number, which reduces recall. This approach fragments the task and can miss important cues that span multiple sentences.

EXAM TIP

When accuracy drops on a specific input style, add small, high quality *few-shot* examples that mirror the hard cases. Match the schema, show edge cases, and keep everything else in the prompt unchanged so the examples isolate the improvement.

REFERENCES

Check out these links for more details:

- [Anthropic Prompt Engineering guide, including using examples to guide outputs](#)